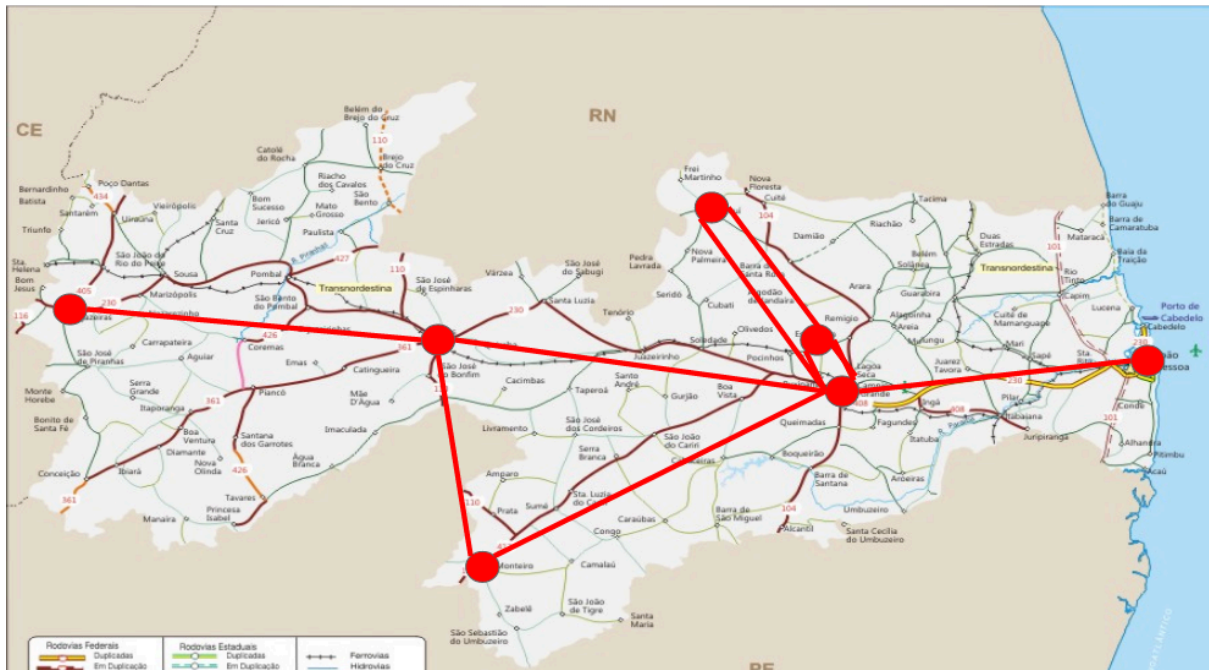


Roteiro 1 - Familiarização com a biblioteca



No exemplo acima, os vértices representam cidades e as arestas indicam se é possível chegar a outra cidade por uma estrada.

Dessa forma, esse grafo pode ser escrito da seguinte forma:

$N = \{J, C, E, P, M, T, Z\}$

$A = \{a1, a2, a3, a4, a5, a6, a7, a8, a9\}$

$g(a1) = JC, g(a2) = CE, g(a3) = CE, g(a4) = CP, g(a5) = CP, g(a6) = CM, g(a7) = CT, g(a8) = MT, g(a9) = TZ$

J: a1

C: a1, a2, a3, a4, a5, a6, a7

1. Para iniciar, construa o grafo da Paraíba usando a biblioteca do Roteiro 0 a partir do módulo disponibilizado [aqui](#). Esse link leva a um repositório do GitHub. Você deve clonar esse repositório usando o git ou baixar o repositório e descompactá-lo em sua pasta de projeto.
2. O repositório que vocês baixaram contém 6 arquivos:
 - a. O arquivo **teste.py** é para vocês tentarem construir o grafo da Paraíba e imprimi-lo na tela.
 - b. O arquivo **meu_grafo_lista_adj.py** é uma “casca” a partir da qual vocês resolverão os problemas de grafos deste roteiro. Nessa casca eu crio a classe **MeuGrafo** que herda da classe **GrafoListaAdjacencia** que está definida dentro da biblioteca. Dessa forma, a classe **MeuGrafo** tem acesso a todos os métodos (funções) e variáveis definidos dentro da classe

GrafoListaAdjacencia. Basta usar a “casca” e implementar as funções que estão em branco.

- c. O arquivo **grafo_lista_adj_test.py** é um arquivo de teste de unidade que você usará para testar as funções que desenvolveu. Eu vou exigir de vocês que aprendam não só a entender os testes mas também a escrevê-los.
3. Crie funções em Python para satisfazer as seguintes funcionalidades:
 - a. Encontre todos os pares de vértices não adjacentes.
 - i. `def vertices_nao_adjacentes(self):`
 - b. Há algum vértice adjacente a ele mesmo? (Retorne True ou False)
 - i. `def ha_laco(self):`
 - c. Há arestas paralelas? (Retorne True ou False)
 - i. `def ha_paralelas(self):`
 - d. Qual o grau de um vértice arbitrário?
 - i. `def grau(self, v=''):`
 - e. Quais arestas incidem sobre um vértice V arbitrário?
 - i. `def arestas_sobre_vertice(self, v):`
 - f. Esse grafo é completo?
 - i. `def eh_completo(self):`